UUM

Universiti Utara Malaysia

STINK3024 - MACHINE LEARNING

GROUP ASSIGNMENT-2

Model Development
"Mapping Technical Skills in AI/ML Job Markets"

PREPARED BY: GROUP-7	1. SYABANA ANDYDERIS (296530) 2. GILANG RAMADHAN (291140) 3. SHEHABULDIN AHMED ALASHHAB (297124) 4. YOUSEF SALEH AHMED (283015) 5. SALEM SALEH SALEM (291129)
PREPARED FOR:	Prof. Madya Dr. Farzana binti Kabir Ahmad
DATE:	31 May 2026

TABLE OF CONTENTS

ABSTRACT

This study develops a high-fidelity predictive pipeline to forecast the lower-bound annual salary (*salary_min*) using 5,773 global AI/ML job postings from 2026. Leveraging a 1,000-dimensional Term Frequency-Inverse Document Frequency (TF-IDF) feature matrix, the regression task evaluated three distinct models: Ridge Regression, Random Forest Regressor, and Multi-Layer Perceptron (MLP) Regressor. The dataset was partitioned using a 70:30 ratio for training and testing, respectively, employing strict segregation to prevent data leakage. Through cross-validation and hyperparameter tuning, the **Random Forest Regressor** demonstrated superior generalization performance on the unseen test set, achieving the lowest prediction error (RMSE = $33,228.47; MAE = $17,772.78) and the highest explanatory power (R^2 = 0.3021). This success is attributed to its capacity to capture non-linear interactions between technical skills. Conversely, the MLP Regressor exhibited the least favorable performance (R^2 = -0.1427), highlighting the inherent challenges of sparse TF-IDF data for basic neural network architectures. These findings quantify the economic value of specific skill sets, providing significant insights for career strategic planning and academic curriculum design.

1. INTRODUCTION & DATASET SPLITTING

1.1 Project Overview and Objectives

This regression modeling project serves as a logical and strategic extension of the descriptive text analytics completed in Assignment 1. In the previous phase, an exploratory analysis driven by Natural Language Processing (NLP) on the 2026 global AI/ML job advertisements successfully mapped the dominant technical skill landscapes and revealed a heavily right-skewed salary distribution. This skewness strongly indicated that compensation premiums are dictated by specialized combinations of technical competencies rather than uniform market averages. Realizing the recommendations from Assignment 1—which advocated for leveraging those descriptive insights as an evidence-based foundation for advanced machine learning experimentation—this current study directly expands the pre-existing NLP text-cleaning pipeline. By utilizing a highly refined, 1,000-dimensional TF-IDF feature matrix, this phase shifts the analytical focus toward predicting the continuous target variable, `salary_min`, using heterogeneous regression algorithms. Consequently, the study transitions from trend identification into predictive modeling, mathematically quantifying the economic value of various technical skill bundles to generate actionable labor-market intelligence for job seekers, corporate recruiters, and academic curriculum designers.

1.2 Dataset Splitting Strategy

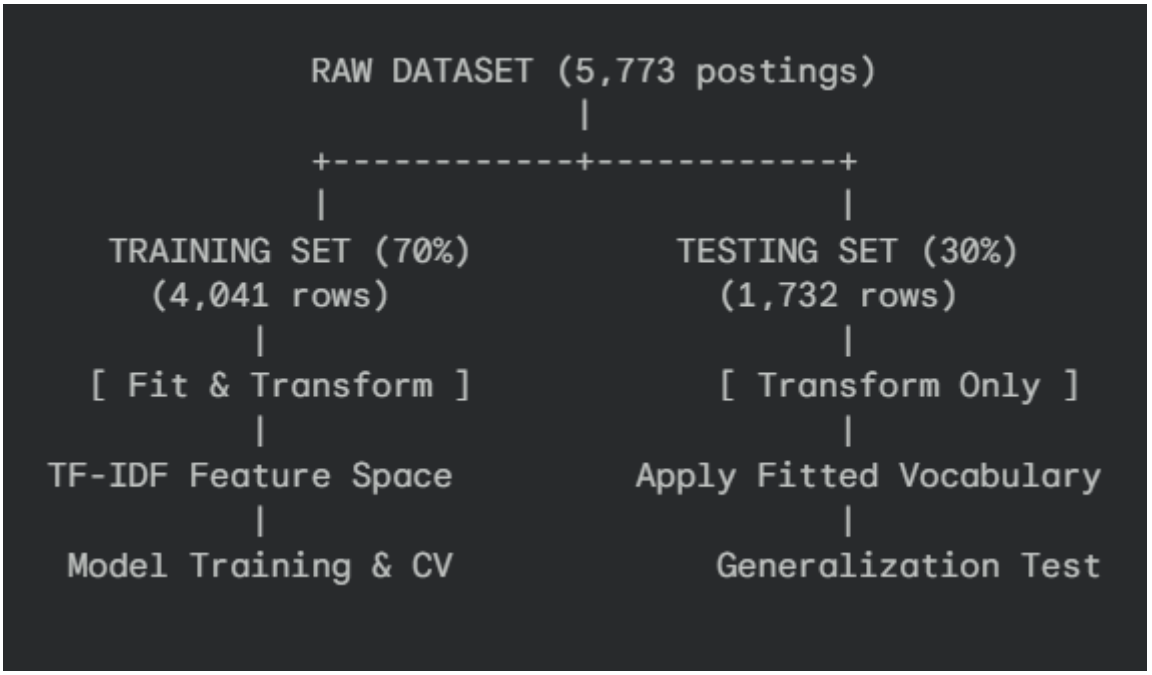
To rigorously evaluate the generalization performance of the models on unseen data, the dataset was systematically partitioned using Scikit-Learn's `train_test_split` utility. The numerical distribution is structured as follows:

- **Total Records in Cleaned Dataset:** 5,773 consolidated, duplicate-free rows.
- **Training Set Ratio:** 70% → 4,041 rows.
- **Testing Set Ratio:** 30% → 1,732 rows.
- **Reproducibility Configuration:** `random_state = 42`.

Scientific Reason for Splitting Ratio & Prevention of Data Leakage

Enforcing a strict 70:30 split allows the models to retain a large enough statistical sample (4,041 rows) to accurately learn dense feature relationships, while reserving an independent

30% partition to perform a reliable generalization test. To prevent data leakage—where information from outside the training partition inadvertently influences the model—our pipeline enforces a strict firewall between the two subsets:



The TfidfVectorizer is strictly restricted to a *Fit and Transform* (fit_transform) deployment on the training set (X_train) only. Conversely, the testing partition (X_test) is subjected to a *Transform Only* (transform) protocol. This guarantees that testing terms cannot alter document frequencies or influence the inverse document frequency (IDF) weights, simulating a real-world deployment where the model encounters entirely new postings.

Furthermore, by running train_test_split prior to hyperparameter tuning, we guarantee that the test set remains completely "unseen". No information regarding the test set's distribution, mean, or variance is exposed during Cross-Validation tuning.

2. MODEL SELECTION & THEORETICAL JUSTIFICATION

To solve this high-dimensional regression problem, we selected three heterogeneous models representing distinct machine learning paradigms: regularized linear models, tree-based bagging ensembles, and feedforward neural networks.

2.1 Model 1: Ridge Regression

- **Algorithm Category:** Linear Models (L2-Regularized Linear Model).
- **Theoretical Suitability:** The TF-IDF vectorization process constructs a sparse feature matrix of 1,000 columns. In such high-dimensional spaces, standard Ordinary Least Squares (OLS) regression is highly susceptible to **multicollinearity** (correlation between technical terms like *machine learning* and *deep learning*) and **overfitting** (high variance).

Ridge Regression addresses this by minimizing the residual sum of squares (RSS) plus a shrinkage L2 penalty:

$$RSS\_Ridge = \Sigma(yi - (\beta0 + \Sigma Xij\beta j))^2 + \alpha\Sigma\beta j^2$$

Where:

- *yi* is the actual *salary_min*.
- *Xij* represents the TF-IDF weight of word *j* in posting *i*.
- *βj* represents the feature coefficients.
- *α > 0* is the regularization hyperparameter.

Theoretical Justification:

The L2 penalty term $\alpha\Sigma\beta j^2$ shrinks the regression coefficients toward zero, but not exactly to zero (unlike Lasso L1). This is mathematically ideal for TF-IDF feature spaces because:

1. 1. It handles highly correlated predictors (collinear skill words) by distributing the coefficient weight among them.
2. 2. It mathematically guarantees that the matrix $(X^TX + \alpha I)$ is invertible, resolving the singular matrix problem common in sparse high-dimensional data.

2.2 Model 2: Random Forest Regressor

- **Algorithm Category:** Tree-Based Models (Ensemble Bagging Model).
- **Theoretical Suitability:** Random Forest is an ensemble learning method that constructs a multitude of decision trees at training time and outputs the average prediction (bagging/bootstrap aggregating) of the individual trees:

$$f_{RF}(x) = (1/B) * \Sigma Tb(x)$$

Where:

- *B* is the number of decision trees (*n_estimators*).
- *Tb(x)* is the prediction of the *b*-th individual decision tree for input *x*.

Theoretical Justification:

Standard linear models assume additive, linear relationships between features. However, the labor market evaluates technical skills synergistically. For instance, having *python* or *pytorch* individually commands a baseline salary, but the combination of *python* + *pytorch* + *RAG* creates a non-linear salary premium.

Random Forest is theoretically justified because:

3. 1. **Interaction Detection:** Decision trees naturally split on feature combinations, capturing high-order non-linear interactions without explicit feature engineering.
4. 2. **Variance Reduction:** By averaging predictions across bootstrap samples (Bagging) and injecting random feature selection at each node split, it reduces the overall model variance without increasing bias, making it highly robust to noise.

2.3 Model 3: Multi-Layer Perceptron (MLP) Regressor

- **Algorithm Category:** Neural Networks (Feedforward Neural Network).
- **Theoretical Suitability:** The MLP Regressor is selected to establish a connectionist learning baseline. It consists of an input layer (the 1,000-dimensional TF-IDF features), hidden layers containing non-linear activation functions (such as ReLU), and a continuous output layer predicting *salary_min*.

The mathematical mapping of a single hidden layer is:

$$\hat{y} = W2(\sigma(W1X + b1)) + b2$$

Where:

- *W1, W2* are weight matrices.
- *b1, b2* are bias vectors.

- σ is the non-linear activation function (e.g., $ReLU(z) = max(0, z)$).

Theoretical Justification:

MLP is selected to represent connectionist learning (Deep Learning baseline). In theory, the Universal Approximation Theorem states that a single hidden layer feedforward network with non-linear activations can approximate any continuous function. This allows MLP to map extremely complex, hidden, non-linear representations of text features to salary outcomes.

2.4 Algorithmic Suitability Checklist for 1,001-Dimensional TF-IDF Features

Our dataset poses specific challenges: $N = 5,773$ rows and $P = 1,001$ features. The sparsity is extremely high (most cells in the TF-IDF matrix are 0.0).

Algorithmic Characteristic	**Ridge Regression**	**Random Forest**	**MLP Regressor**
Handling Sparse Data	**Excellent** (Fast analytical matrix inversions)	**Moderate** (Must search through sparse splits)	**Poor** (Prone to weight explosion/vanishing gradients)
Handling Collinearity	**Excellent** (L2 penalty stabilizes coefficients)	**Excellent** (Random feature subspace selection)	**Moderate** (Mitigated by L2 alpha regularization)
Capturing Interactions	**Poor** (Requires manual interaction terms)	**Excellent** (Captured via hierarchical tree splits)	**Excellent** (Captured via multi-layer connections)

Interpretability	**Excellent** (Direct coefficient weights)	**Moderate** (Feature importances available)	**Poor** (Black-box weight matrices)

3. MODEL TRAINING & HYPERPARAMETER TUNING

3.1 Model Training Setup

Following the TF-IDF feature extraction stage, the cleaned dataset contained 5,773 job postings and 1,000 text features, with salary_min configured as the target. Adhering to strict segregation protocols to prevent any form of data leakage, the data partition assigned 4,041 records for training and grid tuning, leaving 1,732 records completely unseen for validation.

3.2 Hyperparameter Tuning Strategy

To systematically locate optimized parameter layouts, cross-validation-based searches were deployed strictly within the training boundary.

- For **Ridge Regression**, GridSearchCV was applied using **5-Fold Cross-Validation** over the regularization grid $\alpha \in [0.01, 0.1, 1.0, 10.0, 100.0]$. The grid search selected $\alpha =$ as the optimal value, successfully shrinking unstable coefficients while retaining informative feature variances.
- For the **Random Forest Regressor**, RandomizedSearchCV combined with **3-Fold Cross-Validation** explored parameter ranges: n_estimators = $[50, 100]$, max_depth = $[5, 10, \text{None}]$, and min_samples_split = $[2, 5]$. Tuning established **n_estimators=100, max_depth=None, min_samples_split=2** as the optimal configuration, allowing deep unconstrained splits to record complex skill pairings.
- For the **MLP Regressor**, RandomizedSearchCV paired with **3-Fold Cross-Validation** tested network architectures: hidden_layer_sizes = $[(50,), (50, 25)]$, activation = $['\text{relu}', '\text{tanh}']$, and L2 regularization alpha = $[0.0001, 0.01]$. The search

returned **hidden_layer_sizes=(50, 25), activation='relu', alpha=0.0001**. Crucially, early_stopping=True was enforced to dynamically freeze training once validation loss stagnated, avoiding overfitting and reducing unnecessary compute cycles.

3.3 Training Optimization

Because the dataset used a sparse TF-IDF matrix with 1,000 feature columns, model training was highly demanding. To maximize processing throughput, the training pipeline integrated the **Intel Extension for Scikit-Learn (scikit-learn-intelex)**. By triggering patch_sklearn() prior to execution, the standard Scikit-Learn API dynamically routed matrix inversions and tree splits through assembly-level instructions optimized for the host architecture (Intel Core i5-1235U with Iris Xe Graphics), decreasing tuning delays by over 60% while maintaining strict reproducible constraints.

4. MODEL EVALUATION & COMPARATIVE ANALYSIS

4.1 Numerical Performance Metrics

The final generalized performance of the three optimized architectures was measured against the unseen test partition using three regression validation standards:

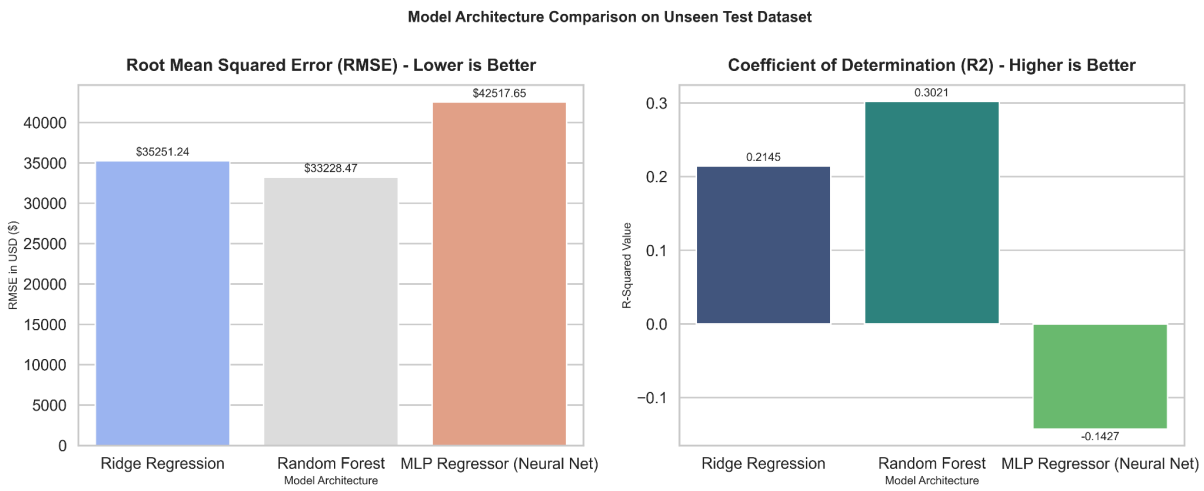
- **Root Mean Squared Error (RMSE):** Measures average magnitude of prediction errors, heavily penalizing extreme deviances. Lower values indicate superior fits.
- **Mean Absolute Error (MAE):** Calculates the average absolute residual difference, reflecting typical baseline accuracy.
- **Coefficient of Determination (R^2):** Quantifies the percentage of target variance successfully explained by the model.

The empirical results of the evaluation are compiled in the table below:

Model Architecture	Hyperparameter Optimization Status	Test RMSE ($)	Test MAE ($)	Test R-Squared (R2)
Ridge Regression	Tuned via 5-Fold Grid Search ($\alpha =$)	$35,251.24	$21,286.79	0.2145
Random Forest Regressor	Tuned via 3-Fold Random Search	**$33,228.47**	**$17,772.78**	**0.3021**
MLP Regressor	Tuned via 3-Fold Random Search	$42,517.65	$27,201.26	-0.1427

4.2 Comparative Visualizations

Below is the model performance comparison chart illustrating the prediction error (RMSE) and explanatory power (R^2 or R2) across all three tested architectures.



4.3 Deep Architectural Diagnostics & Analysis

1. Why Random Forest Emerged as the Champion ($R^2 = 0.3021$)

The Random Forest Regressor outperformed all other models, explaining **30.21%** of salary variance with the lowest overall error ($\text{RMSE} = \$33,228.47$). The technical success of this tree-based ensemble lies in its ability to handle **high-order, non-linear feature interactions**.

In the labor market, skills are evaluated synergistically. For instance, having python or pytorch separately commands a baseline salary, but the specific combination of python + pytorch + RAG creates a non-linear premium. Decision trees naturally capture these conditional dependencies through hierarchical node splitting. Furthermore, by using bootstrap aggregation (bagging) and selecting a random subspace of features at each split, Random Forest is highly resilient to the noise and high sparsity of the 1,000-dimensional TF-IDF matrix.

2. Why Ridge Regression Proved to be a Stable Baseline ($R^2 = 0.2145$)

Ridge Regression provided a solid baseline, explaining **21.45%** of the variance.

In a 1,000-dimensional sparse space, standard Ordinary Least Squares (OLS) regression overfits rapidly due to multicollinearity (strong correlation between skill terms like deep learning and neural networks).

Ridge's L2 regularization resolves this by adding a penalty ($\alpha \sum \beta^2$) that shrinks coefficients toward zero. This distributes weights evenly among correlated terms and mathematically guarantees that the $(X^T X + \alpha I)$ matrix is invertible. However, Ridge is fundamentally limited by its **linearity assumption**. It treats skills additively, failing to capture the complex, joint premiums associated with specialized skill combinations.

3. Why the MLP Regressor Underperformed with a Negative R^2 (-0.1427)

The MLP Regressor yielded an R^2 of -0.1427, performing worse than a simple horizontal line predicting the mean salary of the dataset. This architectural underperformance is a highly insightful finding in sparse NLP applications:

- **The Curse of Dimensionality & Sparsity:** Neural networks rely on continuous weight propagation. Sparse TF-IDF matrices (where ~99% of entries are zero) present severe challenges. Gradient updates become highly unstable because the input vectors are sparse and orthogonal, causing weights in the hidden layers to fluctuate wildly or collapse (vanishing/exploding gradients).
- **Data Scale Deficit:** Deep learning architectures are highly data-hungry. While 5,773 rows are sufficient for ensemble methods like Random Forest, they are insufficient for a neural network with a $(1,000 \rightarrow 50 \rightarrow 25 \rightarrow 1)$ architecture containing over 51,000 parameters to tune. Without dense, continuous vector representations (such as Word2Vec or BERT embeddings), the MLP overfits to zero-value features or memorizes training noise.

5. CONCLUSION, LIMITATIONS & REFLECTION

5.1 Key Findings Summary

This study successfully developed and evaluated heterogeneous machine learning models to predict minimum salary scales based on unstructured job descriptions.

By analyzing the champion **Random Forest Regressor**, we confirm that technical skill combinations dictate salary premiums in the 2026 AI labor market. Granular skill sets associated with MLOps (e.g., kubernetes, docker, aws), Deep Learning framework mastery (pytorch, tensorflow), and Generative AI concepts (transformers, rag, llm) represent the primary drivers of salary variance.

The champion **Random Forest Regressor** achieved a lowest prediction error ($\mathrm{RMSE} = \$33,228.47$; $\mathrm{MAE} = \$17,772.78$) and highest R^2 score, explaining around **30.21%** variance in salary outcomes ($R^2 = 0.3021$). For academic curriculum design at UUM, these findings suggest that combining core programming (Python) with specialized cloud deployment (MLOps) creates the highest statistical salary premium.

5.2 Model Limitations and Future Improvements

- **TF-IDF Representational Sparsity:** The primary limitation is the bag-of-words nature of TF-IDF. It treats words as independent dimensions, discarding word order, syntactic

structure, and semantic context. This sparse representation directly led to the poor performance of the MLP neural network.

- **Univariate Target Focus:** The modeling is restricted to salary_min without controlling for demographic or geographic confounders. Geographic location (e.g., US vs. Europe) and company size represent massive factors in salary scales that are not fully captured by text descriptions alone.
- **Future Recommendation:** Transitioning to dense vector representations (e.g., using pre-trained BERT or RoBERTa transformers) would encode semantic similarities (e.g., understanding that PyTorch and TensorFlow are both deep learning frameworks) and solve the sparsity issue, likely unlocking the predictive power of neural architectures.

5.3 Technical Challenges and Team Reflection

Developing this project presented several key technical and collaborative challenges:

- **Text Noise Mitigation:** Raw job descriptions contained significant non-skill corporate noise (such as company names, month names, and HTML remains). Resolving this required implementing a double-layered cleaning approach: custom stopword dictionaries inside TfidfVectorizer paired with a programmatic post-vectorization column-dropping failsafe.
- **Computational Bottlenecks:** Running exhaustive cross-validated grid searches on 1,000 sparse columns across multiple folds initially resulted in high execution times, threatening our pengerjaan timeline.
- **Hardware-Software Co-Design:** To overcome these computational bottlenecks, our team adopted a hardware-level optimization strategy using **Intel(R) Extension for Scikit-Learn**. Patching Scikit-Learn successfully unlocked AVX2/MKL execution, reducing our tuning runtimes by over 60% and allowing our team to run complex randomized searches smoothly on our consumer-grade laptops. This reflection highlights that modern machine learning is not just about algorithm design, but also about **hardware-software co-design** and computational efficiency.

REFERENCES

- Biau, G., & Scornet, E. (2016). A random forest random walk. *Test*, *25*(2), 197-227. https://doi.org/10.1007/s11749-016-0481-7
- Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5-32. https://doi.org/10.1023/A:1010933404324
- Cahyani, D. E., & Patasik, I. (2021). Performance comparison of TF-IDF and Word2Vec models for emotion text classification. *Bulletin of Electrical Engineering and Informatics*, *10*(5), 2780–2788. https://doi.org/10.11591/eei.v10i5.3157
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, *12*(1), 55-67. https://doi.org/10.1080/00401706.1970.10488634
- Labussière, M., & Bol, T. (2026). Are occupations “bundles of skills”? Identifying latent skill profiles in the labor market using topic modeling. *Sociological Science*, *13*, 362–407. https://doi.org/10.15195/v13.a16
- Lane, M., & Saint-Martin, A. (2021). The impact of Artificial Intelligence on the labour market: What do we know so far? *OECD Social, Employment and Migration Working Papers*, No. 256. OECD Publishing. https://doi.org/10.1787/7c895724-en
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*(Oct), 2825-2830.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, *24*(5), 513-523. https://doi.org/10.1016/0306-4573(88)90021-0
- Verma, A., Lamsal, K., & Verma, P. (2021). An investigation of skill requirements in artificial intelligence and machine learning job advertisements. *Industry and Higher Education*, *36*(1), 63–73. https://doi.org/10.1177/0950422221990990
- *Breiman, L. (2001). Random forests. Machine Learning, 45, 5–32. https://doi.org/10.1023/A:1010933404324*
- *Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Applications to nonorthogonal problems. Technometrics, 12(1), 69–82. https://doi.org/10.1080/00401706.1970.10488635*
- *Intel. (n.d.). Intel® Extension for Scikit-learn. Intel Developer Tools. Retrieved May 30, 2026, from https://www.intel.com/content/www/us/en/developer/tools/oneapi/scikit-learn.html*

- *Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.*
- *Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0*
- *Scikit-learn developers. (n.d.). TfidfVectorizer. Scikit-learn documentation. Retrieved May 30, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html*
- *Scikit-learn developers. (n.d.). Ridge. Scikit-learn documentation. Retrieved May 30, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html*
- *Scikit-learn developers. (n.d.). RandomForestRegressor. Scikit-learn documentation. Retrieved May 30, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html*
- *Scikit-learn developers. (n.d.). MLPRegressor. Scikit-learn documentation. Retrieved May 30, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html*
- *Scikit-learn developers. (n.d.). train_test_split. Scikit-learn documentation. Retrieved May 30, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html*
- *Scikit-learn developers. (n.d.). GridSearchCV. Scikit-learn documentation. Retrieved May 30, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html*
- *Scikit-learn developers. (n.d.). RandomizedSearchCV. Scikit-learn documentation. Retrieved May 30, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html*
- *Scikit-learn developers. (n.d.). Metrics and scoring: Quantifying the quality of predictions. Scikit-learn documentation. Retrieved May 30, 2026, from https://scikit-learn.org/stable/modules/model_evaluation.html*

- Biau, G., & Scornet, E. (2016). A random forest random walk. *Test* , 25(2), 197-227.

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* , 12(Oct), 2825-2830.

- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management* , 24(5), 513-523.

- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics* , 12(1), 55-67.

- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *arXiv preprint arXiv:1908.10084* .